

Contents

Security Review (Full) — LedgerLite (fictional multi-tenant invoicing API)	1
Scope & method	1
Risk summary	1
Findings	2
A-01 — Critical	2
A-02 — Critical	2
A-03 — High	2
A-04 — Critical	3
A-05 — High	3
A-06 — High	3
A-07 — Medium	4
A-08 — Medium	4
A-09 — High	4
A-10 — Medium	5
A-11 — Medium	5
A-12 — High	5
A-13 — High	6
Remediation plan	6
Retest notes	6

Security Review (Full) — LedgerLite (fictional multi-tenant invoicing API)

Deliberately insecure for demonstration and training. Do not deploy. Run locally only.

Prepared: 2026-09-28 · **Tier:** Standard/Advanced · **Target:** targets/tenant-api (baseline)
vs. targets/tenant-api-fixed (retest target)

Scope & method

Same scope as the Starter-tier ARCHITECTURE_REVIEW.md, to the full finding set. Method: manual review of every controller/repository call plus this lab's own **isolation-tester** harness (isolation-tester/), which mechanically proves cross-tenant/cross-role access failures rather than relying on manual testing alone - see isolation-matrix.md (this directory) for the full endpoint×actor matrix. Every LEAK row's own request/response capture is committed under results/isolation-tester/baseline/raw/ (the path is in that row's Evidence column) - DENIED/control-row captures are regenerated locally on each harness run rather than committed, to keep this repo small.

Harness run summary, baseline (from results/isolation-tester/baseline/isolation-matrix.json): 185 non-control probes across every declared endpoint × actor combination (24 additional positive-control probes confirmed legitimate owner access still works), preflight OK, positive controls OK. **86 probes classified LEAK**, confirming findings A-01, A-02, A-03, A-04, A-05, A-06, A-07 mechanically.

Harness run summary, fixed (from results/isolation-tester/fixed/isolation-matrix.json): 124 non-control probes (24 positive controls), **0 probes classified LEAK**, exit code 0.

Risk summary

Severity	Count
Critical	3
High	6
Medium	4

Severity	Count
Low	0
Total	13

Findings

A-01 — Critical

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization

Effort to fix: S

How found: harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceController.java:45`

What's wrong. GET `/invoices/{id}` loads the invoice by primary key alone - it never checks that the invoice belongs to the caller's own tenant.

Why it matters. Any authenticated user from ANY tenant can read any other tenant's invoice by guessing or enumerating sequential ids - full cross-tenant financial data disclosure (line items, totals, customer names).

Exact fix. Load by (id, tenant_id) together - either a repository method `findByIdAndTenantId`, or a Postgres RLS policy comparing tenant_id to a session-local `app.tenant_id` (see targets/tenant-api-fixed). Diff: `git show 2a61da3`.

A-02 — Critical

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization

Effort to fix: S

How found: harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/customer/CustomerController.java:48`

What's wrong. PUT `/customers/{id}` updates the customer row by id alone, with no tenant check on the write path.

Why it matters. Any authenticated user from any tenant can OVERWRITE another tenant's customer record - not just read, but corrupt another company's data.

Exact fix. Same fix pattern as A-01, applied to the write path: `findByIdAndTenantId` before the update. Diff: `git show 2a61da3`.

A-03 — High

OWASP: A01:2021 - Broken Access Control · API3:2023 - Broken Object Property Level Authorization

Effort to fix: M

How found: harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceController.java:59`
- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceCreateRequest.java:8`
- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceResponse.java:6`

What's wrong. The invoice JSON exposes internal fields (tenantId, cost fields) the client should never see or set, and POST `/invoices` lets the client set tenantId directly on create (mass assignment).

Why it matters. A malicious or compromised client can create an invoice tagged as belonging to a DIFFERENT tenant, planting data in another company’s account; the exposed internal fields leak cost/margin data never meant for the API consumer.

Exact fix. Introduce request/response DTOs that never carry tenantId; derive tenantId server-side from the caller’s own token, never from the request body. Diff: `git show 2a61da3`.

A-04 — Critical

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization

Effort to fix: S

How found: harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/customer/CustomerController.java:27`
- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceController.java:33`
- `src/main/java/com/ledgerlite/tenantapi/tenant/TenantContext.java:13`

What’s wrong. The tenant context is resolved from an X-Tenant-Id HTTP header instead of the verified JWT’s own tenant claim.

Why it matters. Any caller can set this header to an arbitrary value and impersonate any tenant for every request - this is the root-cause enabler behind most of the OTHER cross-tenant findings, since it means “which tenant is this request for” is entirely client-controlled.

Exact fix. Resolve tenant_id from the JWT’s own verified claim only; ignore any client-supplied header for this purpose entirely. Diff: `git show 2a61da3`.

A-05 — High

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization

Effort to fix: M

How found: harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/export/ExportController.java:42`
- `src/main/java/com/ledgerlite/tenantapi/export/ExportService.java:16`

What’s wrong. Report exports are written to a shared temp path keyed by a sequential integer id, with no ownership check on download.

Why it matters. An attacker can walk sequential export ids and download OTHER tenants’ exported invoice reports - a bulk, offline variant of the same cross-tenant data leak as A-01.

Exact fix. Check export ownership (tenant_id) before serving; use non-sequential (UUID) export ids as defence in depth. Diff: `git show 2a61da3`.

A-06 — High

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization

Effort to fix: S

How found: harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/dashboard/DashboardController.java:13`

What’s wrong. The dashboard summary cache key does not include the tenant, so cached totals from one tenant are served to the next.

Why it matters. A tenant can see another tenant’s dashboard revenue/invoice totals - cross-tenant data leak via a shared cache, intermittent and timing-dependent (harder to notice than a direct API leak, easy to miss in a manual review).

Exact fix. Key the cache entry on `tenant_id` (e.g. “summary:{tenantId}”), never a constant string. Diff: `git show 2a61da3`.

A-07 — Medium

OWASP: A01:2021 - Broken Access Control · API5:2023 - Broken Function Level Authorization

Effort to fix: S

How found: harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/config/SecurityConfig.java:20`
- `src/main/java/com/ledgerlite/tenantapi/invite/InviteController.java:14`

What’s wrong. POST `/users/invite` is only hidden from the `viewer` role in the UI - the endpoint itself has no server-side role check.

Why it matters. A `viewer`-role user (meant to be read-only) can call the endpoint directly and invite new users into the tenant, including - depending on what role the request specifies - potentially privileged roles.

Exact fix. Add `@PreAuthorize` (or equivalent) role enforcement server-side; never rely on UI-only hiding for an authorization decision. Diff: `git show 2a61da3`.

A-08 — Medium

OWASP: A07:2021 - Identification and Authentication Failures · API2:2023 - Broken Authentication

Effort to fix: M

How found: manual

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/config/SecurityConfig.java:29`
- `src/main/resources/application.yml:27`

What’s wrong. JWT audience is never validated, tokens are long-lived, and there is no refresh-token rotation.

Why it matters. A token issued for a different, unrelated audience/service can be replayed against this API; long-lived tokens with no rotation widen the window a stolen token stays useful.

Exact fix. Validate the `aud` claim against this API’s own identifier; issue short-lived access tokens with refresh rotation and reuse detection. Diff: `git show 2a61da3`.

A-09 — High

OWASP: A05:2021 - Security Misconfiguration · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: tool (gitleaks)

Evidence:

- `src/main/resources/application.yml:8`

What’s wrong. The JWT signing secret and the database password are hardcoded in the committed `application.yml`.

Why it matters. Anyone with read access to the source repository (including a former employee, a leaked backup, or a public-repo mistake) obtains credentials that can forge auth tokens or connect to the database directly.

Exact fix. Read both from environment variables with no hardcoded fallback; the app should fail to start if they're unset, not silently use a default. Diff: `git show 2a61da3`.

A-10 — Medium

OWASP: A04:2021 - Insecure Design · API4:2023 - Unrestricted Resource Consumption

Effort to fix: M

How found: manual

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/auth/AuthController.java:19`
- `src/main/java/com/ledgerlite/tenantapi/export/ExportController.java:17`

What's wrong. Login and invoice export have no rate limiting.

Why it matters. Login is brute-forceable at unlimited speed; export has no protection against being used as a resource-exhaustion or cross-tenant-enumeration accelerant (see A-05).

Exact fix. Add per-identity rate limiting (e.g. Bucket4j) on both endpoints, with a sensible lockout/backoff policy. Diff: `git show 2a61da3`.

A-11 — Medium

OWASP: A09:2021 - Security Logging and Monitoring Failures · N/A - the API Top 10 2023 has no logging/monitoring category; see OWASP 2021 mapping

Effort to fix: M

How found: manual

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/invite/InviteController.java:20`
- `src/main/java/com/ledgerlite/tenantapi/logging/RequestLoggingFilter.java:14`

What's wrong. Role changes and exports are not audited at all, and application logs contain full bearer tokens.

Why it matters. No forensic trail exists for privilege changes or data exports after an incident; logged tokens are themselves a secondary credential leak (anyone with log access can replay them).

Exact fix. Add an `audit_log` table written in the same transaction as the sensitive action; scrub Authorization header values from all log output. Diff: `git show 2a61da3`.

A-12 — High

OWASP: A03:2021 - Injection · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: manual

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceController.java:74`
- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceSearchRepository.java:9`

What's wrong. The invoice search endpoint's `sort` query parameter is concatenated directly into the SQL ORDER BY clause.

Why it matters. Classic SQL injection via the sort parameter - and NOT meaningfully contained by the query’s separately-parameterised WHERE tenant_id clause: a payload like a CASE WHEN subquery in the ORDER BY position turns this into a boolean-blind oracle that can read any table the database role can see, tenant-scoped or not, one bit at a time, and (depending on the JDBC driver’s statement-batching config) may permit stacked statements. Treat this as full database read access, not a tenant-scoped leak.

Exact fix. Whitelist sort against a fixed set of known column names server-side before it ever reaches the query; never string-concatenate user input into SQL. Diff: `git show 2a61da3`.

A-13 — High

OWASP: A10:2021 - Server-Side Request Forgery · API7:2023 - Server Side Request Forgery

Effort to fix: M

How found: manual

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/webhook/WebhookController.java:12`

What’s wrong. The webhook ‘test’ endpoint fetches whatever URL the caller provides, including internal/metadata IP ranges.

Why it matters. Classic SSRF - an attacker can use this endpoint to probe internal network services or reach the cloud metadata endpoint (169.254.169.254) and potentially exfiltrate instance credentials.

Exact fix. Require https, enforce a per-deployment host allow-list, reject IP literals and private/link-local ranges, disable redirects, and validate BEFORE the outbound call is made. Diff: `git show 2a61da3`.

Remediation plan

Sprint	Findings	Rationale
Sprint 1	A-01, A-02, A-04	Critical - active cross-tenant/cross-user data exposure or full compromise
Sprint 2	A-03, A-05, A-06, A-09, A-12, A-13	High - exploitable with realistic effort, real business impact
Sprint 3	A-07, A-08, A-10, A-11	Medium - narrower blast radius or requires a secondary condition

Retest notes

`targets/tenant-api-fixed` is this repo’s own retest target: a separate Maven module/database/Keycloak realm implementing every fix above in commit `2a61da3` (`git show 2a61da3`; see also its own README’s “What changed, per finding” table). Re-running the SAME isolation-tester config against it is the retest evidence - see the “Harness run summary, fixed” line above, computed from `results/isolation-tester/fixed/isolation-matrix.json` (never trust this report’s prose over that generated file - regenerate and compare before relying on this for a real retest sign-off).